



Rust for Safety-critical systems

Introducción:

Este trabajo comenta las ideas principales del podcast SE Radio 690: Florian Gilcher on Rust for Safety-Critical Systems donde Florian Glicher, cofundador de Ferrous Systems y la Rust Foundation, explica qué implica desarrollar software para entornos mission-critical y safety-critical, y por qué Rust puede aportar ventajas relevantes en ese contexto.

Criticidad del proyecto:

Gilcher establece una distinción importante entre los sistemas *mission-critical* y *safety-critical*. Los sistemas *mission-critical* son aquellos cuyo fallo puede provocar pérdidas económicas graves, interrupciones operativas severas o la caída de servicios e infraestructuras esenciales. Por ejemplo, la caída completa de un centro de datos. Por su parte, los sistemas *safety-critical* son aquellos en los que un fallo puede poner en riesgo vidas humanas. No obstante, Gilcher señala que esta distinción es cada vez más difusa, ya que ambos tipos de sistemas tienden a entremezclarse. Por ejemplo, si fallara un centro de datos que da soporte a un hospital, el impacto no sería solo económico u operativo, sino que también podría afectar indirectamente a la seguridad de las personas.

Uso de Rust para sistemas críticos:

Glicher hace especial referencia a que Rust es un muy buen compañero para aquellos que quieran desarrollar alguno de estos sistemas. Esto se debe a que Rust garantiza seguridad en memoria y concurrencia en tiempo de compilación. Estas garantías se logran gracias a dos conceptos fundamentales: propiedad y préstamo. Con propiedad se hace referencia a que siempre está claro que parte del programa posee un recurso y con préstamo a un sistema de referencias seguras con ciclos de vida rastreados por el compilador. Otra de las ventajas que propone Glicher es que Rust no necesita un entorno de ejecución complejo, Rust genera código nativo, así como C, pero seguro. El hecho de no necesitar un entorno de ejecución complejo elimina riesgos como la impredecibilidad por pausas del recolector de basura o la dificultad de certificación. Todo esto combinado hace que Rust sea una buena opción para sistemas de aviación, automoción, medicina...

Rust vs C/C++:

Comparando los dos lenguajes tradicionales usados para los sistemas críticos obtenemos los siguientes resultados:

- Seguridad en memoria: En Rust es garantizada por el compilador en tiempo de compilación. Mientras tanto, en C/C++, es propensa a bugs y requiere herramientas externas costosas
- Verificación: En Rust parte de los fallos se detectan en compilación reduciendo tiempos para las pruebas. Mientras que en C/C++ son necesarios pruebas manuales y pruebas para post-compilación.
- Ecosistema: Rust al ser de cierto modo "moderno" tiene un ecosistema en crecimientos. Mientras que C/C++ tiene un ecosistema maduro que se ha ido perfeccionando con el paso de los años
- Concurrencia: En Rust el compilador previene ciertas condiciones de carrera limitando el aliasing mutable. Mientras que en C/C++ hay un mayor grado de libertad y por ende un mayor riesgo de errores.



Algunos ejemplos de grandes empresas que mejoraron su productividad al usar Rust son Google o Volvo, las cuales llegaron a una productividad de dos a cuatro veces mayor gracias a la reducción de tiempo al revisar código y encuentro y arreglo de bugs.

Ferrocene: Toolchain Certificada en Rust

Ferrocene es una cadena de herramientas para Rust, específicamente diseñada y certificada para sistemas safety-critical. A diferencia de las cadenas de herramientas genéricas como GNU GCC o LLVM oficial, Ferrocene cumple estándares industriales estrictos como ISO 26262 (automoción), IEC 61508 (instrumentación) o IEC 61508 (industrial). Las características que destaca Gliber son:

- Un compilador Rust auditado con trazabilidad completa
- Evidencia documentada para cada optimización
- Soporte para bare-metal (sistemas sin sistema operativo)
- Herramientas de verificaciones integradas

Gilcher también recalca la importancia de la evidencia documentada ya que esta es crucial para la certificación porque mantiene unos beneficios prácticos notorios: las empresas conocen qué está probado y en qué componentes pueden confiar haciendo así que se reduzca drásticamente el tiempo de pruebas de certificación. Evita pruebas innecesarias para código ya validado y acelera la homologación ante reguladores. Gilcher explica que, sin esta documentación detallada, cada empresa debería recertificar todo desde cero, multiplicando costos y tiempo. Con Ferrocene, reutilizan evidencia ya aprobada, pasando directamente a certificar su lógica de negocio. Esto es revolucionario para industrias como automoción y aviación donde la certificación puede tomar años

Limitaciones:

La entrevista también señala que, en entornos críticos, hay que tratar la toolchain como parte del sistema. Por ejemplo: Algunas opciones del compilador están pensadas para comodidad del desarrollador (como la compilación incremental) y pueden aumentar el riesgo de miscompilación; por ello, en builds finales se tienden a desactivar. Aunque Rust reduce muchos fallos, 'unsafe' existe y debe estar justificado, revisado y acotado. La certificación no depende solo del compilador: requiere procesos de calidad, verificación, revisión y evidencia (incluyendo por qué se confía en cada componente).

Casos de uso reales:

- Tock OS: Sistema operativo completo escrito en Rust, usado por Google en varios productos. Lo desarrolló la Universidad de Princeton y ahora se está certificando en Rumania.
- Dispositivo médico: Un equipo médico se implementó totalmente en Rust en solo un mes, cumpliendo todos los requisitos. Actualmente está en proceso de evaluación regulatoria.
- Sudo: Versión moderna del programa Unix clásico, reescrita en Rust por Tweede Golf. Resultó más compacta, rápida y segura. Durante el proceso descubrieron errores en la versión original en C.
- Android (Google): Google integra Rust masivamente en Android, reportando el doble de productividad en desarrollo.
- Volvo: En proyectos mixtos C/Rust, Volvo consiguió entre 2 y 4 veces más velocidad de desarrollo.

Futuro:

Gilcher sostiene que la adopción de Rust en sistemas safety-critical no debe hacerse por moda, sino cuando exista una necesidad real, por ejemplo, mejorar la seguridad de memoria o reducir ciertos riesgos del software. De cara al futuro, considera que Rust puede llegar a convertirse en uno de los lenguajes de referencia en este ámbito, ya que cada vez más proyectos exigen una estrategia clara de memory safety. Aun así, no plantea un reemplazo absoluto de otros lenguajes



como C++, sino una evolución del sector en la que coexistirán varias tecnologías y habrá aprendizaje mutuo entre ellas. Además, también señala que el mercado laboral en torno a Rust está madurando, con ofertas en empresas importantes del sector y con especial interés inicial desde áreas de calidad y validación.